Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Main Future Trends

- Hybrid Integration Platforms (HIP)
- AI-Powered Integration
- Low-Code/No-Code Platforms
- Blockchain for Integration
- Serverless Integration
- Unified Data Integration
- Data Mesh
- IoT Integration

Hybrid Integration Platform (HIP)

- Definition: Combines on-premise and cloud integration tools to handle diverse IT ecosystems.
- Key Features:
  - Legacy system integration.
  - Real-time data synchronization.
  - Flexibility to scale as needed.
- Use Case: Bridging traditional ERP systems with modern SaaS applications like Salesforce.
- Example Tools: MuleSoft, Dell Boomi.

AI

- AI in Software Development: use of intelligent algorithms and techniques to automate, optimize, and enhance various stages of the software lifecycle, from design and coding to testing and maintenance. This includes code generation, intelligent debugging, predictive analysis, and quality assurance.
- AI in System Integration: leveraging AI technologies to connect and harmonize diverse systems, applications, and data sources. This enables seamless communication and interoperability between components, reducing complexity and improving operational efficiency.

Ca' Foscari University of Venice

AI – some use cases

- Intelligent Automation
  - AI tools assist developers by suggesting code snippets, completing functions, and generating entire modules based on functional requirements.
  - AI creates, executes, and analyzes test cases to identify bugs and vulnerabilities faster and more efficiently.
- Predictive Analysis
  - Machine learning models analyze historical data and code to predict areas prone to errors, allowing proactive interventions.
  - AI monitors system performance in real-time, predicting bottlenecks and suggesting improvements.
- Integration and Interoperability
  - AI algorithms automatically map data schemas across disparate systems, simplifying integration tasks.
  - AI orchestrates complex workflows by managing interactions between services and applications without manual intervention.
- Adaptability and Customization
  - Integrated systems can automatically adjust to changing conditions or requirements, thanks to continuous learning and model adaptation.
  - AI tailors' software interfaces and functionalities to individual user preferences and behaviors.

Ca' Foscari University of Venice

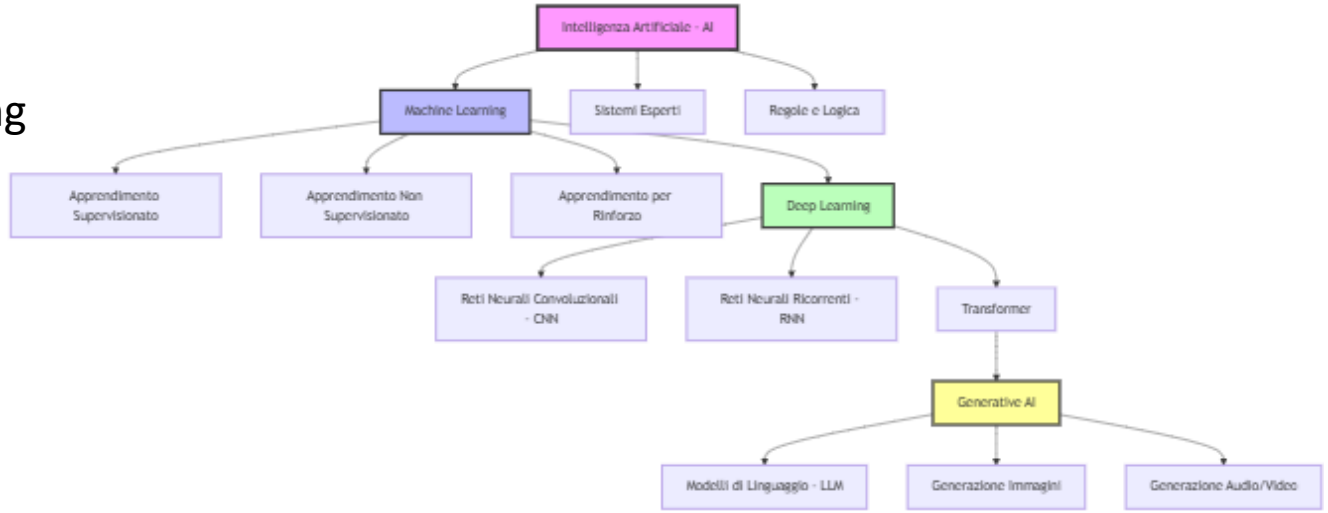
AI – practical examples

- AI-Powered Coding Assistants
  - GitHub Copilot: An AI-driven coding assistant that suggests code and completes functions within the editor, helping developers code faster with fewer errors.
- AI-Powered Integration Platforms
  - MuleSoft Anypoint Platform: Employs AI to automate data mapping and orchestrate integration flows between heterogeneous applications and services.
- Intelligent Testing Tools
  - Testim: Uses machine learning to create, execute, and maintain test cases. It adapts to changes in the application, reducing manual test maintenance efforts.
- Proactive Monitoring and Maintenance
  - Dynatrace: Leverages AI to monitor applications in real-time, detect anomalies, and predict issues before they impact users, providing detailed insights for resolution.
- Intelligent API Integration
  - IBM App Connect: Offers AI-driven integration capabilities for connecting cloud and on-premises applications. AI simplifies complex integrations by automating data mapping and workflow configuration.

Ca' Foscari University of Venice

AI – some theory

- Some definitions
  - Artificial Intelligence: field of computer science focused on creating machines capable of performing task that typically require human intelligence
  - Machine Learning (ML): a subset of AI involving algorithms and statistical models that enable computers to improve their performance on a task through experience
  - Deep Learning: a subset of ML based on artificial neural networks, where algorithms learn from large amounts of data to identify patterns and make decisions
  - Generative AI: AI technologies that can generate new content, ideas, or data that are coherent and plausible.



Ca' Foscari University of Venice

AI – some theory

- Foundation models are large-scale artificial intelligence models that have been trained on vast amounts of data.
- These models are highly versatile and can be adapted to a wide range of tasks and applications.
- Generative AI is one of the applications of foundation models.
- Key aspects:
  - Pretrained on a vast dataset
  - Generalized
  - Adaptable to suit particular needs or tasks
  - Large (trained on extensive data)
  - Self-supervised

- Specific types of foundation model, designed to understand and generate text, are called Large Language Models (LLMs).
- Trained on huge amounts of text, adaptable for different uses involving languages.
- Four types of Generative AI
  - Generative Adversarial Networks (GANs), used for image generation
  - Variational Autoencoders (VAEs), involve in task about compressing and generating high-quality images
  - Transformer Models, for text generation, translation and automated writing
  - Restricted Boltzmann Machines (RBMs), to understand complex data patterns.

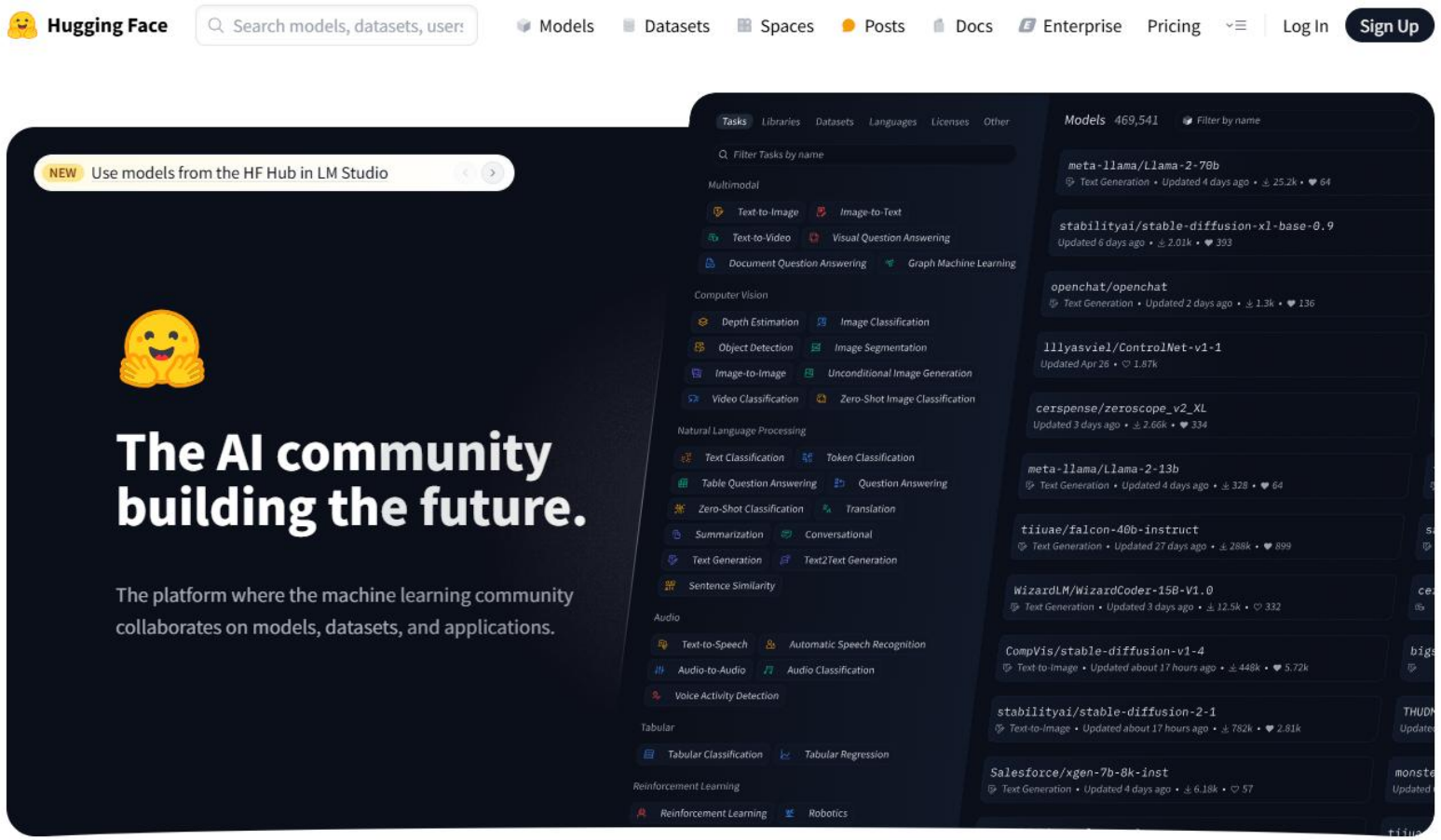
Ca' Foscari University of Venice

AI - LLM

- Commercial/Close
  - OpenAI GPT
    - Best for NLP (Natural Language Processing)
    - Excellent context understanding
    - Robust API
    - Strong generation capabilities
  - Anthropic Claude
    - Greater emphasis on ethics
    - Better understanding of complex instructions
  - Google Gemini
    - Strong Google ecosystem integration
    - Advanced multimodality
- Open-source
  - Meta Llama
    - Good performance
    - Commercial use license
  - Mistral
    - High performance
    - Efficient
    - Good coding capabilities
  - Mixtral 8x7B
    - High performance
    - Computational efficiency
    - Performance comparable to GPT

Ca' Foscari University of Venice

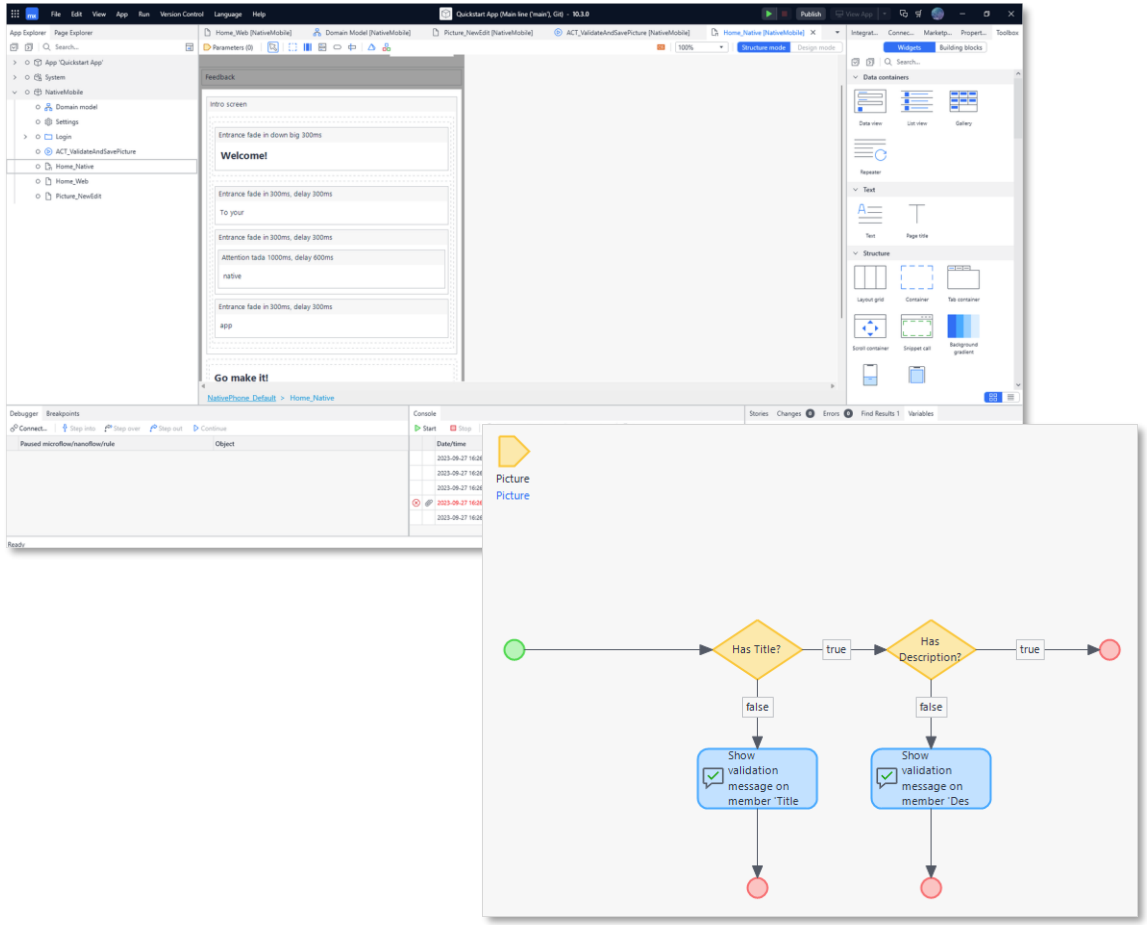
AI – Useful Resource



Ca' Foscari University of Venice

Low-Code/No-Code Platforms

- Definition: Platforms that allow non-developers to create integrations using drag-and-drop interfaces.
- Key Features:
  - Faster deployment cycles.
  - Reduced dependency on IT teams.
  - Pre-built connectors for popular applications.
- Use Case: Integrating CRM data with email marketing tools without custom coding.
- Example Tools: Microsoft Power Automate, OutSystems or Mendix.



Ca' Foscari University of Venice

Video promo as example

Ca' Foscari University of Venice

Low-Code/No-Code Platforms

- The future of low-code and no-code platforms is poised for significant growth and transformation
- Low-code and no-code platforms are increasingly **incorporating artificial intelligence (AI) and machine learning (ML) capabilities**. This integration enables users to build more intelligent and adaptive applications, enhancing automation and data-driven decision-making.
- Traditionally associated with simple app development, low-code and no-code platforms are **evolving to handle more complex applications**. They now offer advanced workflows, seamless integrations, and sophisticated management features, broadening their applicability across various business processes.
- **Larger enterprises are adopting them** to streamline development processes and reduce reliance on extensive IT resources. This shift facilitates rapid prototyping and deployment of applications, fostering innovation and agility within organizations.
- **Empowering non-technical employees**, known as "citizen developers", to create applications is becoming more prevalent. This democratization of app development allows for faster responses to business needs and reduces bottlenecks in IT departments.
- With broader adoption, there is a heightened focus on ensuring that low-code and no-code platforms adhere to **security standards and compliance regulations**. This emphasis is crucial for protecting sensitive data and maintaining trust in the applications developed.

Blockchain for Integration

- Definition: Uses decentralized, immutable ledgers to securely exchange data between systems.
- Key Features:
  - Enhanced data security and transparency.
  - Reduces reliance on intermediaries.
  - Efficient for multi-party transactions.
- Use Case: Supply chain management for tracking goods and verifying authenticity.
- Example Platforms: Hyperledger, Ethereum.

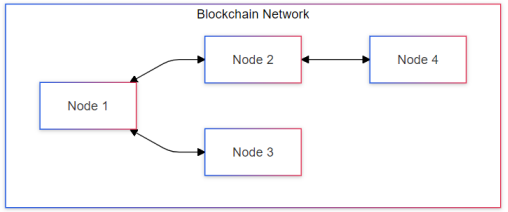
Blockchain what?

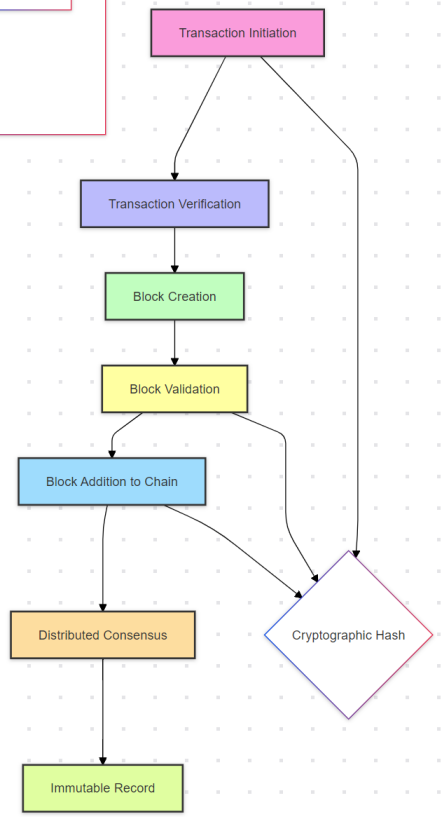
- A blockchain is a decentralized, distributed digital ledger technology that records transactions across multiple computers in a way that ensures the security, transparency, and immutability of data. It allows digital information to be recorded and distributed, but not edited or deleted, creating a chain of blocks that are cryptographically linked and continuously verified by a network of participants.

Ca' Foscari University of Venice

Blockchain what?

- Transaction Initiation
  - A user initiates a transaction
  - Transaction details are prepared
- Transaction Verification
  - Network nodes verify the transaction
  - Check transaction validity
  - Ensure sender has sufficient resources
- Block Creation
  - Verified transactions are grouped into a block
  - Each block contains: Transaction data, Timestamp, Previous block's hash and a Unique block hash
- Block Validation
  - Nodes validate the block
  - Apply consensus mechanisms
  - Common mechanisms
    - Proof of Work (Bitcoin)
    - Proof of Stake (Ethereum)
- Block Addition to Chain
  - Validated block is added to the blockchain
  - Becomes part of the permanent, immutable record
- Distributed Consensus
  - Network nodes agree on the block's validity
  - Prevent double-spending
  - Ensure network security
- Immutable Record
  - Once added, blocks cannot be altered
  - Provides transparency and trust
  - Each block linked cryptographically





Ca' Foscari University of Venice

Blockchain use cases

Finance and Payments

- Bitcoin (BTC) and Ethereum (ETH)
  - As decentralized cryptocurrencies, Bitcoin and Ethereum pioneered blockchain applications. Bitcoin focuses on peer-to-peer payments, while Ethereum supports smart contracts, enabling decentralized applications (DApps).
- Ripple (XRP)
  - Ripple enables real-time cross-border payments for financial institutions. Banks like Santander and American Express use Ripple's blockchain technology to speed up international transactions.
- MakerDAO (DAI)
  - A decentralized finance (DeFi) application offering stablecoins and decentralized lending, built on the Ethereum blockchain.

Supply Chain Management

- IBM Food Trust
  - Major companies like Walmart and Nestlé use IBM's blockchain to trace food products' origins, ensuring transparency and accountability in the food supply chain.
- VeChain
  - A supply chain platform that uses blockchain to verify product authenticity, fight counterfeiting, and enhance transparency in industries like luxury goods and pharmaceuticals.

Blockchain use cases

Healthcare

- MediBloc
  - A decentralized healthcare platform allowing patients to own and control their medical data while providing secure sharing with healthcare providers.
- Guardtime
  - Estonia uses blockchain through Guardtime for national health records, ensuring data security and reducing fraud.

Gaming and Entertainment

- Axie Infinity
  - A blockchain-based play-to-earn game where players own and trade in-game assets (e.g., characters, items) as NFTs.
- Enjin
  - Provides blockchain tools for gaming, enabling developers to tokenize in-game assets, which can then be traded or owned outside the game.

Blockchain use cases

Real Estate

- Propy
  - A blockchain-based platform for buying and selling real estate, offering smart contracts to automate property transactions securely and transparently.
- Ubitquity
  - Used for title deed management and record-keeping, reducing fraud and streamlining property ownership processes.

Identity and Credentials

- Civic
  - A blockchain-based identity verification system that allows users to securely store and share their personal information.
- Spring Labs
  - Utilized for identity verification in financial services, reducing fraud and improving user privacy.

Blockchain use cases

Retail and E-Commerce

- OpenSea
  - The largest decentralized NFT marketplace, enabling users to buy, sell, and trade digital collectibles and artwork.
- Lolli
  - A rewards platform giving Bitcoin cashback for online purchases at major retailers.

Energy

- Power Ledger
  - Uses blockchain for peer-to-peer energy trading, enabling users to buy and sell renewable energy directly.
- WePower
  - A platform connecting renewable energy producers with consumers, offering tokenized energy contracts.

Blockchain use cases

Media and Content

- Brave Browser
  - A web browser that rewards users with Basic Attention Token (BAT) for viewing privacy-respecting ads.
- Audius
  - A decentralized music streaming platform where artists maintain control over their content and revenue.

Charity and Non-Profit

- Giveth
  - A platform using blockchain to provide transparency in charitable donations, ensuring funds reach their intended recipients.
- BitGive
  - Focuses on using Bitcoin and blockchain for tracking charitable donations.

Serverless Integration

- Definition: Integration through serverless computing platforms, which execute code in response to events.
- Key Features:
  - No infrastructure management required.
  - Cost-effective, pay-per-use model.
  - Scalable to handle spikes in data flow.
- Use Case: Real-time notifications triggered by database updates.
- Example Platforms: AWS Lambda, Google Cloud Functions.

Unified Data Integration

- Definition: Real-time, unified data access across multiple systems.
- Key Features:
  - Centralized data view without duplication.
  - Simplified data governance.
  - Improved decision-making with integrated analytics.
- Use Case: Combining marketing, sales, and customer support data for unified reporting.
- Example Tools: Snowflake, Google BigQuery.

Unified Data Integration

- Unified Data Integration (UDI) is a strategic approach to consolidating data from various sources into a unified view that facilitates seamless access, management, and analysis. It is a cornerstone for organizations aiming to leverage data-driven decision-making across complex systems.

Ca' Foscari University of Venice

Unified Data Integration

- Increased Data Sources: Organizations today gather data from diverse sources like databases, cloud applications, IoT devices, and social media.
- Demand for Insights: Decision-makers require integrated and consistent data for business intelligence, analytics, and operational efficiency.
- Challenges Without UDI:
  - Data Silos: Isolated systems leading to inconsistencies.
  - Redundant Efforts: Lack of integration causes repeated tasks and wasted resources.
  - Limited Scalability: Organizations cannot handle growing data demands without a unified system.
- Key Benefits of UDI
  - Enhanced Decision-Making: Unified data offers a single source of truth.
  - Operational Efficiency: Streamlined workflows reduce redundancies.
  - Improved Collaboration: Departments can access and share data seamlessly.
- Examples of UDI in Action
  - Retail: Consolidating inventory and sales data to optimize supply chains.
  - Healthcare: Integrating patient records from multiple clinics for better care.
  - Finance: Real-time risk assessments through unified transaction data.

UDI – Core component

- Data Sources
  - Structured Data: Relational databases (e.g., MySQL, PostgreSQL).
  - Semi-structured Data: XML, JSON files.
  - Unstructured Data: Emails, images, social media posts.
- Integration Processes
  - ETL (Extract, Transform, Load): Extract data from sources, transform it to a standardized format, and load it into a central system.
  - ELT (Extract, Load, Transform): Directly load data into storage (e.g., data lake) and transform it later for analysis.
  - Data Virtualization: Access data without physically moving it to a central location.
- Unified Data Model
  - Provides a standardized schema or framework to ensure compatibility between different systems.

Approaches to UDI

- Centralized Integration
  - Data Warehouses: Aggregated, structured data designed for analytics.
  - Data Lakes: Raw, unstructured or semi-structured data stored for flexible processing.
- Decentralized Integration
  - Data Mesh: Domain-driven approach, where each team handles its data products.
  - Federated Systems: Access data across systems without consolidating it.
- Hybrid Models
  - Combining centralized and decentralized methods to balance flexibility and control.

Ca' Foscari University of Venice

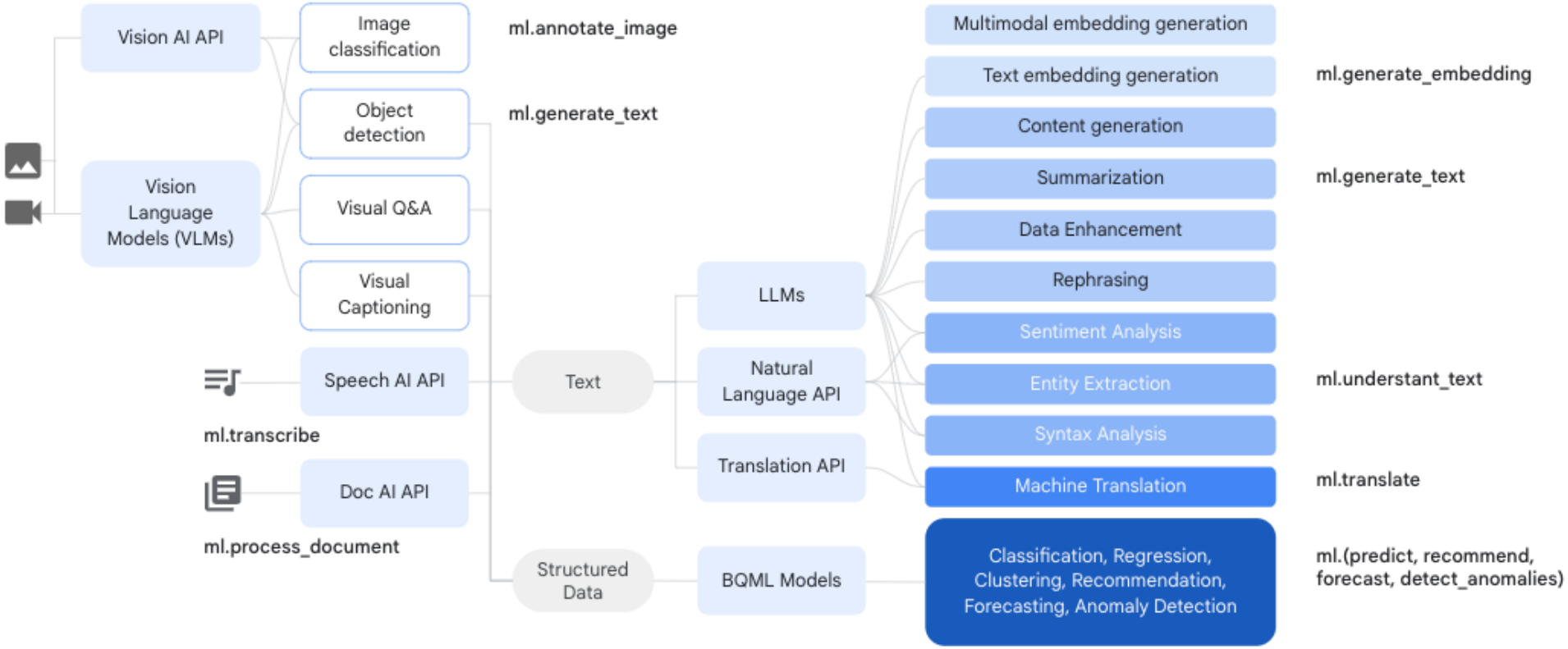
UDI – Technological Enablers

- Middleware Solutions
  - Bridges between systems to ensure seamless communication (e.g., Enterprise Service Buses, API Gateways).
- Integration Platforms
  - Tools like Informatica, Talend, and MuleSoft streamline data extraction, transformation, and loading.
- Cloud Platforms
  - AWS Glue: Simplifies ETL processes.
  - Azure Data Factory: Offers pipelines for data movement and transformation.
  - Google Cloud Dataflow: Real-time and batch data processing.
- Emerging Technologies
  - AI and Machine Learning: Automating data mapping and anomaly detection.
  - Real-Time Data Streaming: Apache Kafka, Apache Flink for processing continuous data streams.
  - Edge Computing: Enabling local data integration for IoT devices.

Ca' Foscari University of Venice

Google Big Query

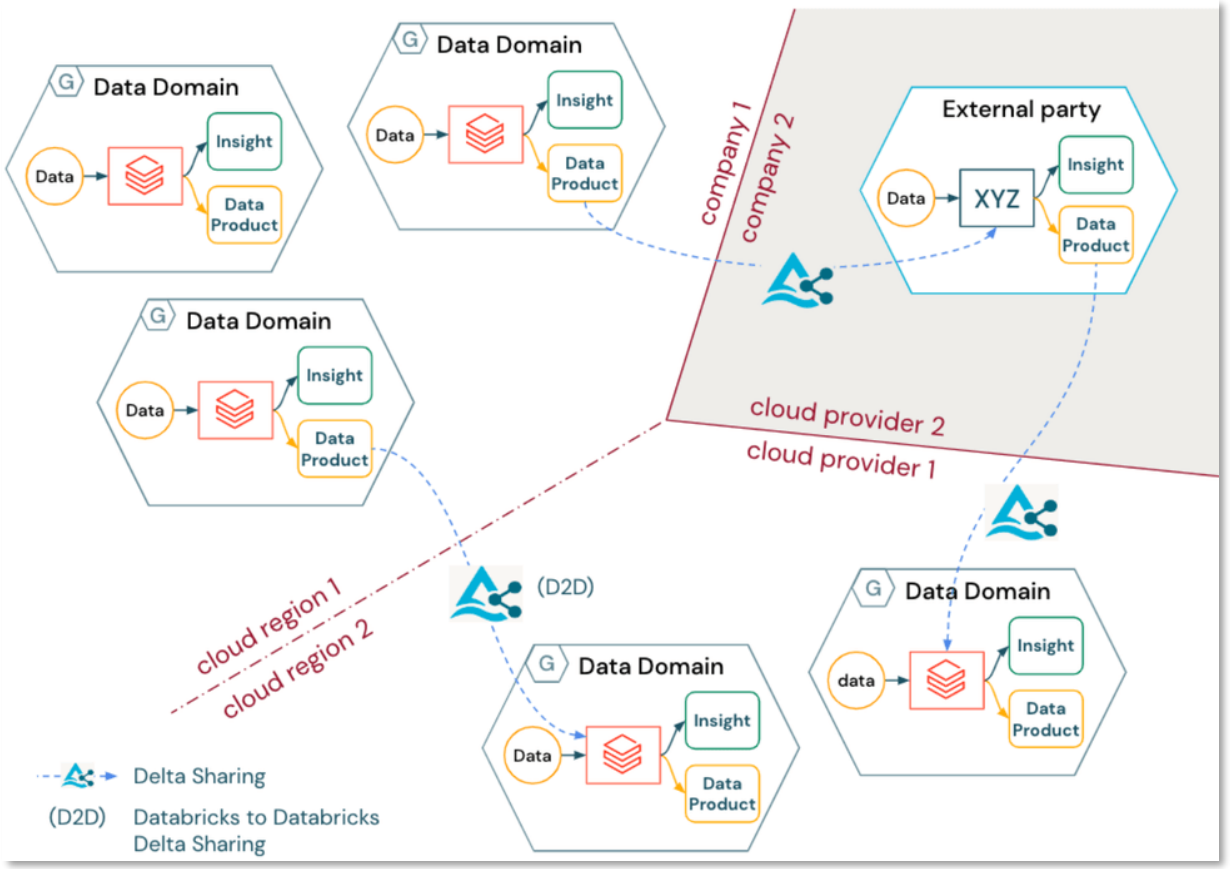
- BigQuery is a fully managed, AI-ready data analytics platform that helps you maximize value from your data and is designed to be multi-engine, multi-format, and multi-cloud.
- https://cloud.google.com/bigquery



Ca' Foscari University of Venice

Data Mesh

- Definition: Decentralized data architecture where each domain manages its data products.
- Key Features:
  - Enhanced modularity and scalability.
  - Domain-specific ownership of data pipelines.
  - Simplifies cross-domain integrations.
- Use Case: Large organizations managing data pipelines across finance, marketing, and operations.
- Example Tools: Data Mesh architecture patterns using Databricks or Snowflake.



Definition of Data Mesh

- Data Mesh is a decentralized approach to data architecture that treats data as a product and assigns ownership of data to the teams that generate and use it. Unlike traditional centralized data solutions (e.g., data warehouses or lakes), Data Mesh focuses on enabling domain-oriented teams to manage, own, and share their data autonomously, fostering scalability, agility, and efficiency in complex organizations.

Ca' Foscari University of Venice

Key Principles of Data Mesh

- Domain-Oriented Data Ownership
  - Data is organized and owned by specific business domains (e.g., marketing, sales, operations), ensuring that the teams closest to the data are responsible for its quality and relevance.
- Data as a Product
  - Teams treat data as a product, ensuring it is discoverable, understandable, reliable, and interoperable for users within and outside the domain.
- Self-Serve Data Infrastructure
  - A platform is provided to enable teams to autonomously handle data ingestion, processing, and serving without relying on centralized IT or data teams. This includes tools for storage, compute, governance, and security.
- Federated Computational Governance
  - Establishes global standards and policies for security, compliance, and interoperability while allowing domains to maintain autonomy.

Data Mesh - Benefits

- Scalability: Decentralization reduces bottlenecks in centralized systems, enabling organizations to scale data operations more effectively.
- Agility: Teams can make faster, more informed decisions without waiting for centralized approvals or resources.
- Improved Data Quality: Domain experts are directly responsible for ensuring the accuracy and reliability of their data products.

Ca' Foscari University of Venice

Comparison with Traditional Architecture

Aspect	Traditional Centralized Approach	Data Mesh
Ownership	Centralized data team	Domain-specific teams
Scalability	Limited by centralized infrastructure	Decentralized and scalable
Focus	Storage and processing optimization	Data as a consumable product
Governance	Top-down	Federated

Example Use Case, E-commerce Platform Adopting Data Mesh

- Domain: Marketing owns campaign data, Operations owns logistics data, and Sales owns transaction data.
- Outcome: Each team provides high-quality, ready-to-use data products, enabling the organization to deliver better customer personalization, optimize supply chains, and improve sales forecasting.

Ca' Foscari University of Venice

zalando

- Zalando, a leading European e-commerce platform, implemented a Data Mesh architecture to address challenges in scaling its data operations. The company needed to manage vast amounts of data across multiple domains, including logistics, customer experience, marketing, and inventory management.

Ca' Foscari University of Venice

zalando

- Domain-Oriented Data Ownership
  - Zalando decentralized its data management by assigning ownership of data to specific business domains.
  - Each domain (e.g., Marketing, Logistics, Customer Service) became responsible for creating and maintaining its own data products.
- Data as a Product
  - Teams treated data as a product by ensuring:
    - Discoverability: Using a data catalog to make datasets easy to find.
    - Quality: Implementing automated tests and monitoring for data reliability.
    - Usability: Providing APIs and clear documentation for external consumers.
  - Example: The Logistics team developed a "Shipment Tracking" data product that other teams could use to track orders in real-time.
- Self-Serve Data Infrastructure
  - Zalando built a self-serve platform to empower teams with tools and templates for managing their data pipelines.
  - Key components included:
    - Data Processing: Apache Kafka for real-time streaming and Spark for batch processing.
    - Storage: AWS S3 for raw data and Google BigQuery for analytics-ready datasets.
    - Orchestration: Apache Airflow for workflow automation.
- Federated Computational Governance
  - Zalando implemented a governance framework to ensure:
    - Compliance with data protection regulations (e.g., GDPR).
    - Standardized schemas and formats across domains for interoperability.
    - Tracking data lineage to understand transformations and usage.

Ca' Foscari University of Venice

zalando

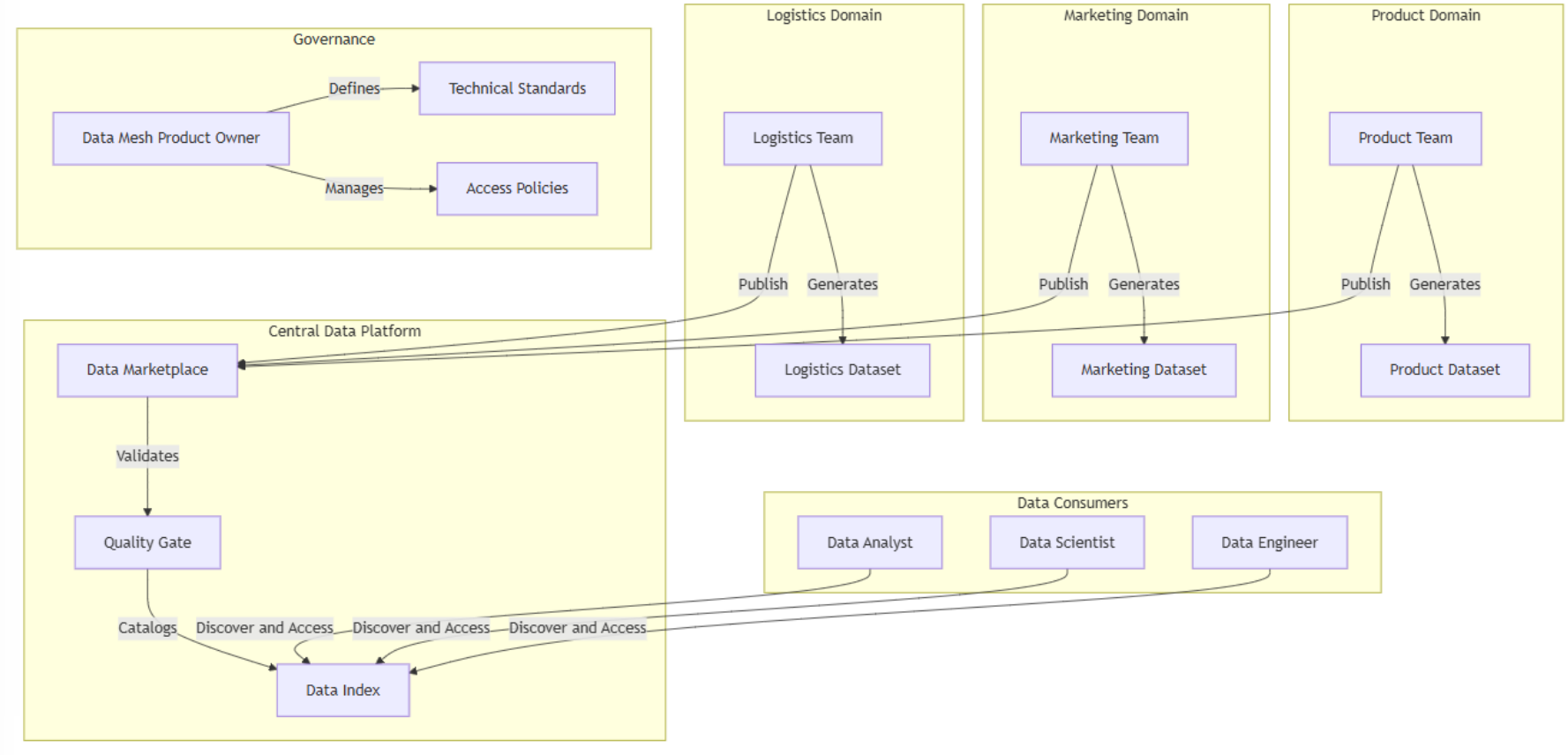
- Improved Scalability: Teams independently managed their data pipelines, reducing the bottleneck on the central data engineering team. The architecture scaled efficiently with growing data volumes.
- Faster Time to Insights: Decentralized ownership enabled teams to deliver insights and analytics much faster without waiting for centralized approvals.
- Better Data Quality and Usability: By treating data as a product, each domain ensured its datasets were well-documented, reliable, and tailored for specific use cases.
- Increased Collaboration: Cross-domain teams collaborated more effectively by leveraging high-quality, easily accessible data products.

- Key learning
  - Cultural Change is Crucial: Training teams to think of data as a product and fostering cross-domain collaboration were critical to success.
  - Start Small, Scale Gradually: Zalando initially piloted Data Mesh in a few domains before scaling it organization-wide.
  - Invest in Infrastructure: Building a robust self-serve platform was vital for empowering teams and ensuring smooth operations.

Ca' Foscari University of Venice

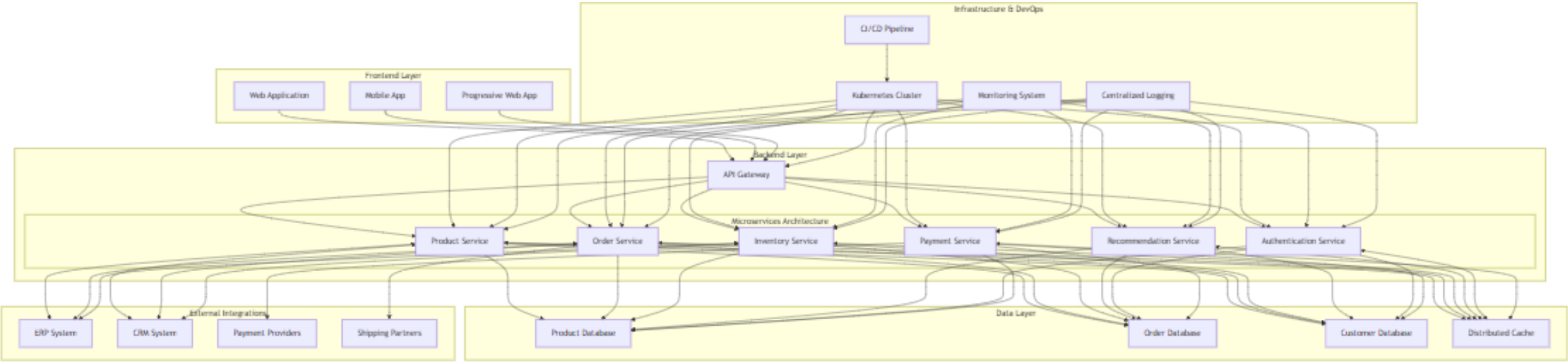
zalando

- Zalando's adoption of Data Mesh helped the company transform its data architecture, making it more scalable, agile, and efficient. It serves as a blueprint for other large organizations seeking to overcome similar challenges in data management.



Ca' Foscari University of Venice

zalando



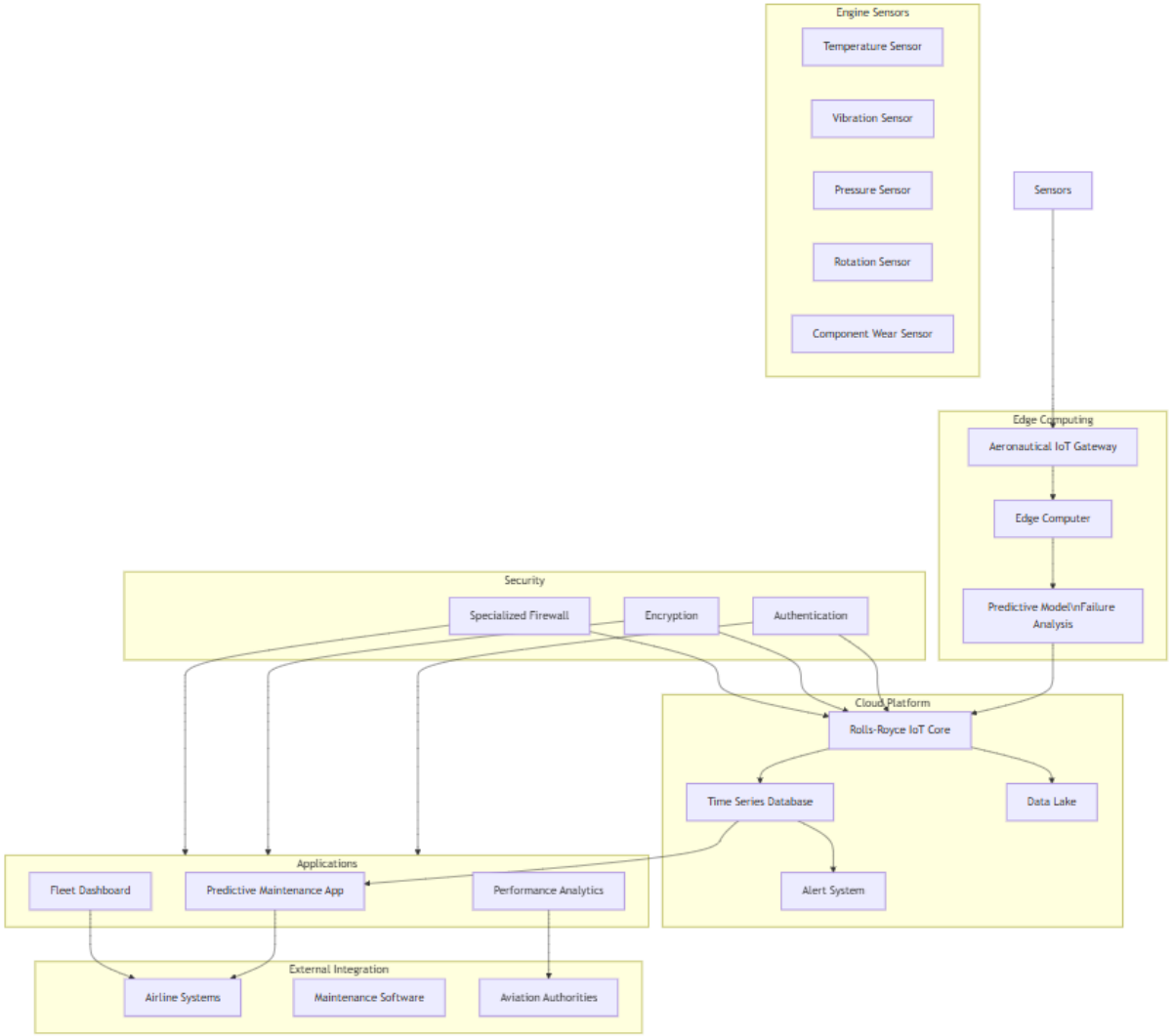
https://engineering.zalando.com/
https://github.com/zalando/engineering-principles

- In March 2015, we have adopted this set of principles for tech and architecture:
  - microservices
  - API First
  - REST
  - Cloud
  - Software as a Service (SaaS)
- "We prefer loosely coupled services. They are more resilient when it comes to remote dependency failures. We aim to develop autonomous isolated services that can be independently deployed and that are centered around defined business capabilities."

- How to build a loosely-coupled system
  - Asynchronous Communication
  - Service Degradation (react to remote dependency failures by degrading a service until the remote dependency is working again)
  - Use Low-Tech Coupling (Communication should be done over interoperable protocols like HTTP instead of, for example, RMI)
  - RESTful APIs with JSON payload
  - Microservices with small size

IoT Integration

- Definition: Connects IoT devices with enterprise systems to enable seamless data flow.
- Key Features:
  - Support for IoT protocols like MQTT and CoAP.
  - Real-time data collection and processing.
  - Edge computing for faster decision-making.
- Use Case: Smart factories monitoring equipment performance in real-time.
- Example Platforms: Azure IoT Hub, AWS IoT Core.

Ca' Foscari University of Venice

Theoretical Example